Compte rendu du TP 5
Réalisé par Meriam Boukraa

Exercice 1:

Voici le code à compléter:

```python
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader


# 1. Transformations des images pour ResNet50
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])


# 2. Chargement du dataset Flickr30k
dataset = datasets.Flickr30k(
    root="/mnt/ssd/flickr30k/flickr30k-images",
    ann_file="/mnt/ssd/flickr30k/results_20130124.token",
    transform=transform
)
```

```python
# 3. Séparation Train (80 %) / Test (20 %)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size

train_dataset, test_dataset = torch.utils.data.random_split(
    dataset,
    [train_size, test_size],
    generator=torch.Generator().manual_seed(42)
)


# 4. Création des DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    num_workers=4
)
```

Réponse à la question 1.a:

Pourquoi utiliser 224 × 224 ? : ResNet50 est couramment utilisé avec des images de cette taille, notamment dans sa configuration pré-entraînée sur ImageNet. Le redimensionnement permet de former des lots homogènes, car les tenseurs d'un même batch doivent avoir les mêmes dimensions. Il simplifie également le calcul et garantit une entrée de taille uniforme. Conserver les résolutions originales compliquerait le regroupement des images et augmenterait potentiellement le coût mémoire et calculatoire.

Résultat de l'exécution:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP5 (main)
$ python dataset.py
Nombre total d'exemples : 31783
Nombre d'exemples d'entraînement : 25426
Nombre d'exemples de test : 6357
Dimensions du batch d'images : torch.Size([32, 3, 224, 224])
Nombre de légendes dans le batch : 5
Première légende : ('A group of people are holding up drinks and pointing to the camera .', 'A medical procedure room fi
lled with high-tech life support systems and an operator in the background .', 'A brown dog is running very fast with hi
s tongue out .', 'A middle-aged man in a white hate , navy sweater , and khaki pants is putting a golf ball .', 'A baldi
ng man pours a bottle of wine into a tumbler while watched by a woman in a yellow shirt .', 'A child stands on a hill ab
ove a village holding a piece of cloth and papers .', 'Outdoor workers repair a sidewalk using a jackhammer in the Unite
d Kingdom .', 'A young , white man in blond matted hair simulates surfing in the evening for a crowd of people .', 'A ma
n in a leather jacket stands on a ladder by an image of a smiling lady holding a turkey dinner .', 'Two cars parked on t
he side of a street with seven people doing different tasks on the sidewalk .', "Several men walk in front of a building
 marked ' Bass Pro Shops Outdoor World ' .", 'Cheerleader on top of a cheerleading pyramid , girls are looking up at her
 one is in the background doing the same cheer .', 'A woman with short black hair in a blue t-shirt holds a baby in pink
```

Exercice 2:

Voici le code à compléter:

```python
import torch
import torch.nn as nn
from torchvision import models


# 1. Chargement de ResNet50 pré-entraîné
resnet_pretrained = models.resnet50(
    weights=models.ResNet50_Weights.DEFAULT
)


# 2. Suppression des deux dernières couches (AdaptiveAvgPool2d et Linear)
# On veut conserver les dimensions spatiales pour l'attention (pas un vecte>
cnn_features = nn.Sequential(
    *list(resnet_pretrained.children())[:-2]
)


# 3. Gel des poids (Transfer Learning)
for param in cnn_features.parameters():
    param.requires_grad = False
```

Réponses aux questions:

1. Pourquoi geler les poids de ResNet50 ?

Le gel des poids de ResNet50 avec requires_grad = False présente deux avantages principaux :

- Raison matérielle : cela réduit la consommation de mémoire et les calculs nécessaires pendant l'entraînement, car il n'est pas nécessaire de calculer et de stocker les gradients pour les paramètres du réseau convolutif. L'entraînement est ainsi plus rapide et moins coûteux en ressources GPU.
- Raison algorithmique : ResNet50 possède déjà des caractéristiques visuelles pertinentes grâce à son pré-entraînement sur ImageNet. Geler ses poids permet de conserver ces représentations et de limiter le risque de surapprentissage, tout en se concentrant sur l'apprentissage du décodeur LSTM et du mécanisme d'attention.

2. Quelle est la dimension de sortie de cnn_features ?

Pour un tenseur d'entrée de forme (32, 3, 224, 224), la sortie de cnn_features est de forme (32, 2048, 7, 7).

- 32 : nombre d'images dans le batch.
- 2048 : nombre de canaux de caractéristiques produits par le dernier bloc convolutionnel de ResNet50.
- 7 × 7 : dimensions spatiales des cartes de caractéristiques.

Ces 49 positions spatiales par image seront utilisées par le mécanisme d'attention pour identifier les régions visuelles pertinentes lors de la génération des légendes.

Exercice 3:

Voici le code de la classe AttentionModule à compléter:

```
)

# Concaténation des caractéristiques de l'image et de l'état du RNN>
concat_features = torch.cat(
    (features, hidden_state_expanded),
    dim=1
)

# Calcul des scores (batch_size, 1, 7, 7) -> on retire la dimension>
attention_scores = self.attention_layer(
    concat_features
).squeeze(1)

# Application du Softmax sur les 49 régions (7x7)
attention_weights = F.softmax(
    attention_scores.view(attention_scores.size(0), -1),
    dim=1
)
attention_weights = attention_weights.view(
    attention_scores.size(0),
    1,
    attention_scores.size(1),
    attention_scores.size(2)
)

# Calcul du vecteur de contexte par somme pondérée sur les dimensio>
context_vector = torch.sum(
    torch.sum(features * attention_weights, dim=3),
    dim=2
)

return context_vector, attention_weights
```

Réponse à la question 3.a:

Le context_vector est une représentation visuelle de l'image obtenue par une somme pondérée des caractéristiques de ses 49 régions, permettant au modèle de se concentrer sur les zones les plus pertinentes à chaque étape de génération de la légende ; il résout ainsi la limitation du Seq2Seq de base, qui repose sur un vecteur de contexte fixe et peut perdre des informations importantes lorsque la séquence est longue.

Question 3.b:

Voici la partie du code à compléter pour la classe LSTMWithAttention:

```
            # Calcul du vecteur d'attention contextuel
            att_t, _ = self.attention_module(features, h_t)

            # Concaténation de l'attention, du mot actuel, et de l'état cac>
            concat = torch.cat((att_t, x_t, h_t), dim=1)

            # Equations du LSTM
            i_t = torch.sigmoid(self.lin_i(concat))
            f_t = torch.sigmoid(self.lin_f(concat))
            g_t = torch.tanh(self.lin_c(concat))
            o_t = torch.sigmoid(self.lin_o(concat))

            # Mise à jour de la mémoire et de l'état caché
            c_t = f_t * c_t + i_t * g_t
            h_t = o_t * torch.tanh(c_t)

            hidden_seq.append(h_t.unsqueeze(1))

        hidden_seq = torch.cat(hidden_seq, dim=1)

        return hidden_seq, (h_t, c_t)
```

Résultat de l'exécution:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP5 (main)
$ python model.py
Shape des images : torch.Size([32, 3, 224, 224])
Shape des caractéristiques : torch.Size([32, 2048, 7, 7])
Tous les paramètres sont gelés : True
Shape du contexte : torch.Size([2, 2048])
Shape des poids d'attention : torch.Size([2, 1, 7, 7])
Somme des poids : tensor([1.0000, 1.0000], grad_fn=<SumBackward1>)
Shape de la séquence LSTM : torch.Size([2, 5, 256])
Shape de l'état caché final : torch.Size([2, 256])
Shape de l'état mémoire final : torch.Size([2, 256])
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP5 (main)
$
```

Les tests confirment que le CNN extrait les caractéristiques visuelles, que l'attention pondère les régions de l'image et que le LSTM traite la séquence avec les dimensions attendues.

Exercice 4:

Voici la classe ImageCaptioningModel à compléter:

```python
class ImageCaptioningModel(nn.Module):
    def __init__(self, cnn_model, embedding_model, lstm_hidden_dim):
        super(ImageCaptioningModel, self).__init__()
        # Backbone ResNet : extraction des caractéristiques visuelles
        self.cnn = nn.Sequential(*list(cnn_model.children())[:-2])
        # Geler les paramètres du CNN pré-entraîné
        for param in self.cnn.parameters():
            param.requires_grad = False

        self.attention = AttentionModule(
            feature_dim=2048,
            hidden_dim=lstm_hidden_dim
        )
        self.embeddings = embedding_model
        vocab_size = embedding_model.weight.shape[0]
        emb_dim = embedding_model.weight.shape[1]

        self.lstm = LSTMWithAttention(
            emb_dim,
            lstm_hidden_dim,
            self.attention,
            2048
        )
        # Prédiction du mot suivant
        self.fc_out = nn.Linear(lstm_hidden_dim, vocab_size)
    def forward(self, images, captions):
        cnn_features = self.cnn(images)
        emb = self.embeddings(captions)

        lstm_outputs, (hidden_state, _) = self.lstm(
            emb, cnn_features
        )

        output = self.fc_out(lstm_outputs)
        return output
```

Résultat de l'exécution:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP5 (main)
$ python model.py
Computing vocabulary...
Number of words: 20324
Vocabulary size: 30810
Embedding dimensions: 300
Embeddings trainable: False
Shape des entrées texte : torch.Size([2, 5])
Shape des sorties : torch.Size([2, 5, 30810])
Taille du vocabulaire : 30810
Shape des images : torch.Size([32, 3, 224, 224])
Shape des caractéristiques : torch.Size([32, 2048, 7, 7])
Tous les paramètres sont gelés : True
Shape du contexte : torch.Size([2, 2048])
Shape des poids d'attention : torch.Size([2, 1, 7, 7])
Somme des poids : tensor([1.0000, 1.0000], grad_fn=<SumBackward1>)
Shape de la séquence LSTM : torch.Size([2, 5, 256])
Shape de l'état caché final : torch.Size([2, 256])
Shape de l'état mémoire final : torch.Size([2, 256])
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP5 (main)
$
```

Le test confirme que le modèle complet fonctionne correctement. Les légendes en entrée sont représentées par des séquences de 5 tokens, et le modèle produit, pour chaque token, un score associé à chaque mot du vocabulaire. La sortie a donc la forme (2, 5, V), où 2

correspond au nombre d'exemples testés, 5 à la longueur des séquences et V à la taille du vocabulaire.

Voici le code de la boucle d'entrainement dans le fichier train.py:

Réponses aux questions:

1. lancement de l'entrainement sur Slurm:

```
Epoch 12/15
Validation loss : 3.2602
Prediction : a man in to a head a man in a a . . . . . .
Target     : a woman talking on her and a man shopping at a small shop . </s
> </s> </s> </s> </s>

Epoch 13/15
Validation loss : 3.2611
Prediction : a man in to a side a man in a a . . . . . .
Target     : a woman talking on her and a man shopping at a small shop . </s
> </s> </s> </s> </s>

Epoch 14/15
Validation loss : 3.2602
Prediction : a man in to a head a man in a a . . . . . .
Target     : a woman talking on her and a man shopping at a small shop . </s
> </s> </s> </s> </s>

Epoch 15/15
Validation loss : 3.2591
Prediction : a man in to a head a man in a a . . . . . .
Target     : a woman talking on her and a man shopping at a small shop . </s
> </s> </s> </s> </s>
[compute] [job 15174] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP5 (mai
n)
$
```

2.

À l'époque 1, la légende générée est peu cohérente et contient de nombreuses répétitions. À l'époque 10, la phrase commence à présenter une structure plus reconnaissable, avec des mots comme « a man » et « head », mais elle reste répétitive et ne décrit pas correctement l'image. La loss de validation diminue de 3,947 à 3,261, ce qui montre que le modèle apprend, mais l'amélioration de la qualité des légendes reste limitée.

3.

Le planificateur StepLR réduit le taux d'apprentissage de 0,001 à 0,0001 après 5 époques, puis à 0,00001 après 10 époques. Cette décroissance permet au modèle d'effectuer des mises à jour plus fines en fin d'entraînement. Si le taux d'apprentissage restait constant à 0,001, les mises à jour pourraient être trop importantes, entraînant des oscillations autour du minimum et limitant la convergence. Cette réduction est particulièrement pertinente avec Adam, qui utilise le momentum et adapte les mises à jour à partir des estimations des gradients : elle complète ce mécanisme en limitant l'amplitude des ajustements et en favorisant une convergence plus stable.